

DBConnection.java:

```
package com.shopping.util; import java.sql.Connection; import java.sql.DriverManager;
import java.sql.SQLException; public class DBConnection {
private static final String URL ="jdbc:mysql://localhost:3306/onlineshoppingdb" + "?useSSL=false" +
"&serverTimezone=UTC"
+ "&allowPublicKeyRetrieval=true";private static final String USER = "shopuser"; private static final String
PASSWORD = "ShopUser@123";static { try { Class.forName("com.mysql.cj.jdbc.Driver"); } catch
(ClassNotFoundException e) {throw new RuntimeException( "MySQL JDBC Driver not found. " + "Make sure
mysql-connector-j is added to the project.", e );}}
public static Connection getConnection() throws SQLException { return DriverManager.getConnection(URL,
USER, PASSWORD);
}}
```

Order.java:

```
package com.shopping.model; import java.math.BigDecimal; import java.sql.Date; public class Order {
private int orderId; private String customerName; private String productName; private int quantity;
private BigDecimal price; private BigDecimal totalAmount; private Date orderDate; private String address;
public Order() { }public Order(String customerName, String productName, int quantity, BigDecimal price,
BigDecimal totalAmount,Date orderDate, String address) { this.customerName = customerName;
this.productName = productName; this.quantity = quantity; this.price = price; this.totalAmount = totalAmount;
this.orderDate = orderDate; this.address = address;}
public int getOrderId() { return orderId; } public void setOrderId(int orderId) { this.orderId = orderId; }
public String getCustomerName() { return customerName; }
public void setCustomerName(String customerName) { this.customerName = customerName; }
public String getProductName() { return productName; }
public void setProductName(String productName) { this.productName = productName; }
public int getQuantity() { return quantity; } public void setQuantity(int quantity) { this.quantity = quantity; }
public BigDecimal getPrice() { return price; } public void setPrice(BigDecimal price) { this.price = price; }
public BigDecimal getTotalAmount() { return totalAmount; }
public void setTotalAmount(BigDecimal totalAmount) { this.totalAmount = totalAmount; }
public Date getOrderDate() { return orderDate; }
public void setOrderDate(Date orderDate) { this.orderDate = orderDate; }
public String getAddress() { return address; }
public void setAddress(String address) { this.address = address; } }
```

OrderDAO.java:

```
package com.shopping.dao; import com.shopping.model.Order; import com.shopping.util.DBConnection;
import java.sql.Connection; import java.sql.PreparedStatement; import java.sql.ResultSet;
import java.sql.SQLException; import java.util.ArrayList; import java.util.List; public class OrderDAO {
public boolean addOrder(Order order) {
String sql =
"INSERT INTO orders " + "(customer_name, product_name, quantity, price, "
+ "total_amount, order_date, address) " + "VALUES (?, ?, ?, ?, ?, ?, ?)";
try (Connection con = DBConnection.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
ps.setString(1, order.getCustomerName()); ps.setString(2, order.getProductName());
ps.setInt(3, order.getQuantity()); ps.setBigDecimal(4, order.getPrice());
ps.setBigDecimal(5, order.getTotalAmount()); ps.setDate(6, order.getOrderDate());
ps.setString(7, order.getAddress()); int rowsInserted = ps.executeUpdate(); return rowsInserted == 1;
} catch (SQLException e) {
System.out.println("===== DATABASE ERROR =====");
System.out.println("Message: " + e.getMessage()); System.out.println("SQL State: " + e.getSQLState());
System.out.println("Error Code: " + e.getErrorCode()); e.printStackTrace();return false;}
public List<Order> getAllOrders() {
List<Order> orderList = new ArrayList<Order>();
String sql ="SELECT order_id, customer_name, product_name, "
+ "quantity, price, total_amount, order_date, address " + "FROM orders " + "ORDER BY order_id DESC";
try (Connection con = DBConnection.getConnection(); PreparedStatement ps = con.prepareStatement(sql);
ResultSet rs = ps.executeQuery()) {
while (rs.next()) {
Order order = new Order(); order.setOrderId(rs.getInt("order_id"));
order.setCustomerName(rs.getString("customer_name"));
}
```

```

order.setProductName(rs.getString("product_name")); order.setQuantity(rs.getInt("quantity"));
order.setPrice(rs.getBigDecimal("price")); order.setTotalAmount(rs.getBigDecimal("total_amount"));
order.setOrderDate(rs.getDate("order_date")); order.setAddress(rs.getString("address"));
orderList.add(order);} catch (SQLException e) {System.out.println("===== FETCH ORDERS ERROR
====="); e.printStackTrace();}return orderList;}}

```

AddOrderServlet.java:

```

package com.shopping.controller; import com.shopping.dao.OrderDAO; import com.shopping.model.Order;
import java.io.IOException; import java.math.BigDecimal; import java.sql.Date;
import javax.servlet.ServletException; import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet; import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse; @WebServlet(name = "AddOrderServlet",
urlPatterns = {"/AddOrderServlet"}) public class AddOrderServlet extends HttpServlet { @Override
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException,
IOException {
response.sendRedirect(request.getContextPath() + "/orderForm.jsp");}
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException,
IOException {
request.setCharacterEncoding("UTF-8");try {
String customerName = request.getParameter("customerName");
String productName = request.getParameter("productName");
String quantityText = request.getParameter("quantity"); String priceText = request.getParameter("price");
String orderDateText = request.getParameter("orderDate");
String address = request.getParameter("address");if (customerName == null ||
customerName.trim().isEmpty()) { showError(request, response, "Customer name is required."); return;}
if (productName == null || productName.trim().isEmpty()) { showError(request, response, "Product name is
required."); return;}
if (quantityText == null || quantityText.trim().isEmpty()) { showError(request, response, "Quantity is required.");
return;}
if (priceText == null || priceText.trim().isEmpty()) { showError(request, response, "Price is required."); return;}
if (orderDateText == null || orderDateText.trim().isEmpty()) { showError(request, response, "Order date is
required."); return;}
if (address == null || address.trim().isEmpty()) { showError(request, response, "Address is required."); return;}
int quantity = Integer.parseInt(quantityText.trim());
if (quantity <= 0) { showError(request, response, "Quantity must be greater than zero."); return; }
BigDecimal price = new BigDecimal(priceText.trim());if (price.compareTo(
BigDecimal.ZERO) < 0) { showError(request, response, "Price cannot be negative."); return;}
Date orderDate = Date.valueOf(orderDateText.trim());
BigDecimal totalAmount = price.multiply(BigDecimal.valueOf(quantity));Order order =
new Order(customerName.trim(), productName.trim(), quantity, price, totalAmount, orderDate,
address.trim());
OrderDAO dao = new OrderDAO(); boolean success = dao.addOrder(order);if (success) {
response.sendRedirect(request.getContextPath() + "/viewOrders.jsp");} else {
showError(request, response, "Failed to place the order. " + "Please check the server console.");}
} catch (NumberFormatException e) {showError(request, response, "Please enter a valid quantity and price.");}
} catch (IllegalArgumentException e) {showError(request, response, "Please enter a valid order date.");} catch
(Exception e) {
e.printStackTrace();showError(request, response, "An unexpected error occurred. " + "Please check the server
console.");}}
private void showError(HttpServletRequest request, HttpServletResponse response, String message)throws
ServletException, IOException { request.setAttribute("errorMessage",
message);request.getRequestDispatcher("/orderForm.jsp").forward(request, response);}

```

orderForm.jsp:

```

<%@ page contentType="text/html;charset=UTF-8" language="java" %> <!DOCTYPE html> <html> <head>
<meta charset="UTF-8"> <title>Add Order</title>
<style>* { box-sizing: border-box; } body { margin: 0; font-family: Arial, sans-serif; background: #f3f6fa; }
.container {
width: 520px; margin: 50px auto; background: white; padding: 35px; border-radius: 12px;
box-shadow: 0 5px 20px rgba(0,0,0,0.12);}

```

```

h2 { text-align: center; color: #222; margin-bottom: 25px; }
label { display: block; margin-top: 15px; margin-bottom: 6px; font-weight: bold; color: #333; } input,
textarea { width: 100%; padding: 11px; border: 1px solid #ccc; border-radius: 6px; font-size: 14px; }
textarea { height: 90px; resize: vertical; } input:focus, textarea:focus { outline: none; border-color: #007bff; }
.submit-btn {
width: 100%; margin-top: 25px; padding: 13px; border: none; border-radius: 6px; background: #007bff;
color: white; font-size: 16px; font-weight: bold; cursor: pointer; }
.submit-btn:hover { background: #0056b3; }
.view-btn { display: block; margin-top: 12px; text-align: center; padding: 11px; background: #28a745; color:
white;
text-decoration: none; border-radius: 6px; }
.error { padding: 12px; margin-bottom: 15px; background: #ffe6e6; color: #c00; border: 1px solid #ffb3b3;
border-radius: 6px; } </style>
</head> <body> <div class="container"> <h2>Add Order</h2>
<%String errorMessage = (String) request.getAttribute("errorMessage"); if (errorMessage != null) {%>
<div class="error"> <%= errorMessage %> </div><% %>
<form action="<%= request.getContextPath() %>/AddOrderServlet" method="post">
<label for="customerName"> Customer Name </label>
<input type="text" id="customerName" name="customerName" required>
<label for="productName"> Product Name </label>
<input type="text" id="productName" name="productName" required>
<label for="quantity"> Quantity </label>
<input type="number" id="quantity" name="quantity" min="1" required> <label for="price"> Price </label>
<input type="number" id="price" name="price" min="0" step="0.01" required>
<label for="orderDate"> Order Date </label> <input type="date" id="orderDate" name="orderDate" required>
<label for="address"> Address </label> <textarea id="address" name="address" required></textarea>
<button type="submit" class="submit-btn"> Submit Order </button></form>
<a href="<%= request.getContextPath() %>/viewOrders.jsp" class="view-btn"> View Orders</a>
</div> </body> </html>

```

viewOrders.jsp:

```

<%@ page import="java.util.List" %> <%@ page import="com.shopping.dao.OrderDAO" %>
<%@ page import="com.shopping.model.Order" %>
<%@ page contentType="text/html; charset=UTF-8" language="java" %> <!DOCTYPE html> <html> <head>
<meta charset="UTF-8"> <title>View Orders</title>
<style> body { margin: 0; padding: 40px; font-family: Arial, sans-serif; background: #f3f6fa; }
.container { width: 95%; margin: auto; background: white; padding: 30px; border-radius: 12px;
box-shadow: 0 5px 20px rgba(0,0,0,0.1); }
h2 { text-align: center; color: #222; } table { width: 100%; border-collapse: collapse; margin-top: 25px; } th,
td { padding: 12px; border: 1px solid #ddd; text-align: center; } th { background: #007bff; color: white; }
tr:nth-child(even) { background: #f7f7f7; }
.add-btn { display: inline-block; margin-top: 20px; padding: 11px 18px; background: #28a745; color: white;
text-decoration: none; border-radius: 6px; }
.empty { text-align: center; padding: 30px; color: #777; }
</style></head> <body> <div class="container">
<h2>All Orders</h2> <% OrderDAO dao = new OrderDAO(); List<Order> orders = dao.getAllOrders(); %>
<% if (orders.isEmpty()) { %> <div class="empty"> No orders found. </div><% %> } else { %><table><tr>
<th>Order ID</th> <th>Customer</th> <th>Product</th> <th>Quantity</th> <th>Price</th>
<th>Total</th> <th>Order Date</th> <th>Address</th></tr>
<% for (Order order : orders) { %><tr>
<td> <%= order.getId() %> </td> <td> <%= order.getCustomerName() %> </td>
<td> <%= order.getProductName() %> </td> <td> <%= order.getQuantity() %> </td>
<td> ₹<%= order.getPrice() %> </td> <td> ₹<%= order.getTotalAmount() %> </td>
<td> <%= order.getOrderDate() %> </td> <td> <%= order.getAddress() %> </td></tr><% %></table><% %>
<a href="<%= request.getContextPath() %>/orderForm.jsp" class="add-btn"> Add New Order</a></div>
</body> </html>

```

index.jsp:

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %> <!DOCTYPE html> <html> <head>
<meta charset="UTF-8"> <title>Online Shopping</title>
<style>body { margin: 0; font-family: Arial, sans-serif; background: #f3f6fa; }

```

```
.container {width: 500px; margin: 120px auto; text-align: center; background: white; padding: 45px;
border-radius: 15px; box-shadow: 0 5px 20px rgba(0,0,0,0.12);}
h1 { color: #222; } p { color: #666; } a {display: inline-block; margin: 10px; padding: 12px 25px; color: white;
background: #007bff;
text-decoration: none; border-radius: 6px;}a:hover { background: #0056b3; }</style></head> <body> <div
class="container">
<h1>Online Shopping</h1> <p>Welcome to the Online Shopping Order System</p>
<a href="orderForm.jsp"> Add Order </a> <a href="viewOrders.jsp"> View Orders </a> </div> </body> </html>
```

Output:

Add Order

Customer Name
Deepika

Product Name
laptop

Quantity
1

Price
50000

Order Date
09/09/2026

Address
sathyamangalam, erode

[Submit Order](#)

[View Orders](#)

All Orders

Order ID	Customer	Product	Quantity	Price	Total	Order Date	Address
3	Deepika S	laptop	1	₹50000.00	₹50000.00	2026-09-09	sathyamangalam, erode
2	Deepika	laptop	1	₹50000.00	₹50000.00	2026-09-09	sathyamangalam, erode
1	bfbdf	fgbfgf	10	₹563856.00	₹5638560.00	2026-09-08	fewgewhdfjedhfkhdaj

[Add New Order](#)